

Rands Social Ecommerce

Product Requirements Document (PRD) and Technical Implementation Blueprint

Target market	Nigeria
Commerce categories	Clothing, footwear, electronics, and adjacent retail verticals
Document purpose	Concept validation, delivery planning, and engineering execution
Document owner	Ade / Rands project team
Primary stack	Next.js, NestJS, PostgreSQL, ORM, Redis, Docker, CI/CD, Prometheus/Grafana, automated tests, idempotent payment flows

Executive summary. Rands is a trust-led social ecommerce marketplace for Nigeria. It combines seller storefronts, product-linked social feed posts, protected checkout, manual delivery quote approval, payout controls, and an internal ledger so buyers can purchase without fear of non-delivery while sellers still benefit from social discovery and conversation-driven selling.

1. Product Overview

Rands is a marketplace-led social ecommerce platform built for Nigerian commerce behaviour. Sellers already acquire demand through TikTok, Facebook, Instagram, and WhatsApp, but the current purchase path is fragmented: discovery happens on social media, trust happens informally, payment may go directly to the seller, and buyers fear non-delivery. Rands centralises the transaction by combining storefronts, shoppable content, protected checkout, order-state visibility, and controlled payout release.

The immediate differentiation is not just another marketplace. It is a trust orchestration platform for social commerce: sellers retain their ability to market visually and conversationally, while buyers gain a structured order lifecycle, clearer recourse, and reduced fear of losing money.

- Give sellers a storefront plus social surfaces such as posts, short videos, and product-linked feed entries.
- Let buyers transact on-platform rather than paying sellers directly on informal channels.
- Keep payment under platform control until delivery, return, or dispute rules allow release.
- Provide a mini finance view for sellers and platform operators: gross sales, fees, pending funds, available funds, payouts, disputes, and growth trends.
- Make commercial policy configurable from admin: commission mode, split-payment mode, subscription mode, withdrawal windows, return periods, and feature toggles.

2. Problem Statement and Market Context

Nigerian commerce has strong social-discovery behaviour. Merchants frequently market on TikTok, Facebook, Instagram, and WhatsApp because those channels generate attention and conversation quickly. The weakness appears at transaction time: identity assurance is limited, fulfilment evidence is weak, pricing can be changed in chat, and direct payment to sellers creates fear of fraud or non-delivery.

Rands addresses that trust gap by making the platform the system of record for payment, order states, delivery confirmation, returns, disputes, and payout eligibility. This means Rands should optimise for the realities of social commerce rather than copy a traditional catalogue-only marketplace. Feed, chat, seller identity, order status, and payout policy are not secondary features; they are the core trust infrastructure.

3. Users and Personas

Persona	Needs	Main concerns	Rands response
Buyer	Find genuine products, pay safely, track order, and raise issues if an item is wrong or undelivered.	Fraud, non-delivery, weak refunds, unclear delivery fees.	Protected checkout, structured delivery update flow, buyer confirmation, and dispute process.
Seller / Merchant	Showcase products, acquire traffic, communicate with buyers, and receive payouts reliably.	Slow sales, weak visibility, payout uncertainty, chargebacks, and admin burden.	Storefront, feed posts, chat, analytics, and configurable payout and policy controls.
Platform admin / operations	Control risk, monitor growth, manage fees, moderate content, and handle disputes.	Fraud, poor reconciliation, weak policy enforcement, and support burden.	Admin toggles, ledger and payout engine, reporting, moderation, and operational dashboards.

4. Product Principles

- Trust before speed. Checkout, payout, and disputes must be correct before they are fast.
- Social-native commerce. Feed, posts, seller profiles, comments, and chat should drive conversion rather than sit beside ecommerce as an afterthought.
- Policy-driven operations. Return windows, payout holds, and commercial models must be configurable rather than hard-coded.
- Modular by design. Features such as feed, chat, subscriptions, and seller tools should be enabled per tenant, category, or rollout wave through feature flags.
- Observable and auditable. Every money movement and important state transition must be traceable.

5. Functional Scope (PRD)

5.1 Seller onboarding and merchant account

- Account registration with email/password at launch; OTP and social sign-in can follow later.
- Seller onboarding with KYC/business profile, payout bank details, store branding, support contact, and policy acknowledgement.
- Storefront page with seller bio, ratings later, catalog, latest posts, and active policies.
- Admin review and approval workflow for new sellers and sensitive categories.

5.2 Catalogue and storefront

- Product creation with title, description, price, images, optional video, category, variants, stock status, weight, and order handling time.
- Seller-managed product links that can be attached to feed posts.
- Product moderation and soft-delete/archive workflows.
- Future-ready fields for brand, authenticity verification, serial numbers, or size guides.

5.3 Social commerce layer

- Feed where sellers can publish image and video posts linked to one or more products.
- Buyer engagement through comments at launch, with likes and follows later.
- Shoppable post cards that route directly to product detail or seller storefront.
- Moderation controls for post takedown, comment review, and abuse handling.

5.4 Buyer-seller communication

- On-platform chat between buyer and seller for pre-sale and post-order communication.
- System guardrails: mask sensitive data where needed, preserve message logs for dispute review, and keep key order changes outside chat in structured workflows.
- WhatsApp notifications for order milestones, while keeping the canonical order record inside Rands.

5.5 Checkout, delivery quote, and payment

Because external delivery integration is not yet in scope, the delivery fee flow should be two-step:

- Buyer selects product and begins checkout.
- System creates an Awaiting Delivery Quote order draft.
- Seller reviews destination and calculates delivery fee manually.
- Buyer is notified, reviews the fee, and accepts or declines.

- Once accepted, the system creates a final payable amount and sends the buyer through Paystack checkout.
- Payment success moves the order into Paid / Awaiting Fulfilment after verification.

5.6 Orders, fulfilment, returns, disputes, and payout release

- Order states: draft, awaiting delivery quote, quote sent, quote accepted, payment pending, paid, seller preparing, shipped/manual in transit, delivered pending buyer confirmation, completed, dispute open, refunded, cancelled.
- Seller manually updates fulfilment milestones and uploads evidence where appropriate.
- Buyer can confirm receipt. If buyer confirms and no dispute exists, funds start or finish release rules.
- If buyer does not confirm within the defined period, the platform may auto-progress according to policy.
- Return and dispute windows are admin-configurable by category and seller tier.
- Seller payout is determined by internal ledger state, not just by raw transaction status.

5.7 Seller finance dashboard

- Gross order value, platform fees, Paystack fees where allocated, pending balance, available balance, paid-out balance, refunds, and disputes.
- Order-level ledger breakdown showing why an amount is pending, reserved, released, refunded, or blocked.
- Payout request flow subject to payout window and seller risk controls.
- Basic charts for GMV, orders, conversion, repeat buyers, and payout trends.

5.8 Admin platform

- Feature toggles by environment, rollout wave, or seller tier.
- Commercial mode toggle: commission, subscription, hybrid, and selective split-payment usage.
- Return policy days, seller withdrawal windows, dispute SLAs, category moderation rules, and media controls.
- Operational dashboards for orders, payout liabilities, reconciliation exceptions, fraud flags, and system health.

6. Non-functional Requirements

Area	Requirement
Security	Server-side payment initialization only; strict secret management; webhook verification; RBAC; audit logs; encryption at rest where appropriate.
Reliability	Idempotent payment and payout flows; retry-safe job processing; dead-letter handling for failed notifications and retries.
Performance	Fast feed and product browsing; cache hot reads in Redis; isolate media delivery via CDN/object storage.
Observability	Structured logs, metrics, traces where possible, Prometheus scraping, Grafana dashboards, and alerting on payment or ledger failures.
Maintainability	Modular codebase, typed contracts, test automation, CI/CD gates, migration discipline, and feature flags.
Compliance-readiness	Record retention, consent and policy acceptance tracking, payout and dispute auditability.

7. Recommended Operating Model for Money Movement

The most important architecture choice is how Rands handles buyer protection. The recommended default is platform-controlled collection plus an internal ledger rail. In practical terms, the buyer pays the platform through Paystack, Rands verifies the payment, and Rands decides when seller balances move from pending to available based on delivery, return, and dispute policy.

This approach supports the trust promise better than treating split settlement as the default flow. Split payments are useful in some commercial arrangements, but they are not the same thing as policy-controlled holding and release.

8. Why Paystack Split Payments Should Not Be the Default Trust Mechanism

Paystack split payments are designed to share settlement for a transaction between one or more parties. That is useful for commission-sharing or marketplace revenue allocation, but a split configuration does not by itself implement an escrow-like release model where funds remain under platform policy control until delivery conditions are satisfied.

For Rands, buyer protection depends on order-state governance and a ledger-controlled payout release model. Therefore, Paystack split payments should be positioned as an optional monetisation or settlement configuration, not as the default trust mechanism.

9. Monetisation Models

- Commission model: the platform charges a percentage or flat fee on successful orders.
- Subscription model: sellers pay monthly or tiered subscription for storefront access or premium features.
- Hybrid model: lower commission plus seller subscription or premium tooling fees.
- Admin-configurable switches should allow the commercial mode to be selected without code changes, subject to safe rollout rules.

10. Payment Architecture with Paystack

Use Paystack primarily for transaction collection, verification, refunds, optional subscription billing, and possibly dedicated virtual accounts later for bank-transfer-heavy experiences.

Recommended payment flow:

- Create order draft and price breakdown in Rands first.
- Initialize the transaction from the backend only; do not expose the secret key client-side.
- Attach Rands references in metadata: order_id, seller_id, buyer_id, pricing version, and commercial mode.
- Send buyer to hosted or inline Paystack checkout.
- On callback, never trust the browser alone. Verify via webhook and/or verify endpoint before marking order paid.
- Store processor reference, amount in subunits, currency, channel, and raw webhook payload for auditability.
- Use the refunds API for full or partial refunds when disputes or fulfilment failures require reversal.

11. Internal Ledger Rail

The internal ledger is central to the trust model. Even if the first version is operationally simple, it should be implemented with accounting discipline rather than ad-hoc numeric columns.

Ledger concept	Purpose
Platform cash clearing	Represents collected money awaiting allocation or external reconciliation.
Seller pending balance	Seller earnings not yet available due to delivery, return, or dispute window.
Seller available balance	Amount eligible for withdrawal or automatic payout.
Platform revenue	Commission, subscription revenue, or other platform fees.
Payment fee reserve	Tracks processor fees and fee responsibility.
Refund reserve / dispute reserve	Holds amounts earmarked for possible reversal.
Payout payable	Approved seller amount scheduled for disbursement.

Minimum ledger events:

- payment_collected
- payment_verified
- fee_accrued
- seller_pending_allocated
- delivery_confirmed
- hold_released
- refund_initiated
- refund_completed
- payout_requested
- payout_approved
- payout_sent
- payout_reversed

Implementation note: each ledger event should write immutable entries with references to the order, transaction, actor, and policy version used. Balance tables can exist for fast reads, but immutable entries remain the source of truth.

12. Delivery, Return, and Dispute Policy Logic

- Per-category or global return window in days.
- Seller withdrawal window separate from buyer return window if needed.
- Manual delivery confirmation supported by seller update plus buyer confirmation or timed auto-resolution.
- Evidence attachments for delivery or disputes.
- Outcome engine that can release funds, partially refund, fully refund, or re-hold seller balances.

13. Data Model (first-cut entities)

Domain	Key entities
Identity	User, Session, Role, Permission, OTPChallenge, SocialIdentity (later)
Merchant	SellerProfile, SellerKYC, BankAccount, Storefront, SubscriptionPlan, SellerEntitlement

Catalog	Product, ProductVariant, ProductMedia, Category, InventorySnapshot
Social	Post, PostMedia, PostProductLink, Comment, ModerationAction
Communication	Conversation, Message, MessageAttachment
Orders	Cart, Order, OrderItem, DeliveryQuote, FulfilmentEvent, ReturnRequest, DisputeCase
Payments	PaymentIntent, PaystackTransaction, WebhookEvent, RefundRecord
Finance	LedgerAccount, LedgerEntry, BalanceSnapshot, PayoutRequest, PayoutBatch
Configuration	FeatureFlag, PolicyRule, FeeRule, NotificationTemplate

14. API and Backend Patterns

- Use NestJS modules with DTO validation, guards, interceptors, and clear domain services.
- Use ORM migrations carefully; maintain migration review in CI.
- Wrap external processor calls in an integration layer rather than scattering Paystack logic across controllers.
- Use idempotency keys for payment initialization, webhook processing, refund requests, and payout actions.
- Route long-running or retryable operations through background jobs backed by Redis queues.

15. Frontend Implementation (Next.js)

- App Router architecture with separate route groups for buyer, seller, and admin surfaces.
- Server actions or API routes for trusted write operations where suitable, but keep payment secrets fully server-side.
- Reusable UI primitives for product cards, feed posts, quote acceptance, order timelines, ledger summaries, and dispute states.
- Media optimisation strategy for image and video delivery, with CDN/object storage and progressive loading.
- Feature-flag-aware rendering to hide incomplete modules safely.

16. Reliability, Observability, and Security Controls

Control area	Recommended control
Webhook security	Validate signatures or source expectations, preserve raw payload, deduplicate repeated events, and reconcile verified status before value release.
Idempotency	Store per-operation idempotency key plus response hash to prevent duplicate charges or duplicate ledger writes.
Secrets	Use environment secret manager; never place Paystack secret keys in client code or mobile builds.
Metrics	Payment success rate, callback-to-verification lag, webhook failures, payout queue depth, dispute rate, ledger imbalance checks, and quote acceptance conversion.
Alerts	High failed-payment spikes, unverified success callbacks, stuck pending orders, payout failures, and reconciliation exceptions.
Auditability	Admin changes to policy, fees, and toggles must produce immutable audit logs.

17. Suggested Delivery Roadmap

Phase	Objective	Indicative scope
Phase 1 - Marketplace trust MVP	Prove protected checkout and payout hold model.	Auth, seller onboarding, catalog, storefront, manual delivery quote, Paystack checkout, verification, order states, internal ledger, seller dashboard basics, and admin policies.
Phase 2 - Social layer	Increase demand generation and conversion.	Feed, post-linked products, comments, share links, richer storefront content, and notifications.
Phase 3 - Communication and operations	Reduce friction and support burden.	Chat, moderation workflows, dispute tooling, refund operations, seller analytics, and payout operations.
Phase 4 - Commercial expansion	Monetise more intelligently.	Subscriptions, hybrid pricing, premium seller tools, selective split-payment mode, and DVA-led transfer experiences later.

18. Key Product Risks and Mitigations

- Fake sellers or poor fulfilment behaviour. Mitigate through onboarding review, tiered trust limits, delayed payout for new sellers, and dispute evidence collection.
- Operational burden from manual delivery updates. Mitigate through strict status flows, required evidence fields, and later logistics integrations.
- Ledger inaccuracies. Mitigate through immutable ledger events, reconciliation jobs, and finance-focused test coverage.
- Feed or chat abuse. Mitigate through moderation tools, reporting workflows, rate limits, and content policies.
- Policy confusion. Surface delivery quote acceptance, return windows, and payout conditions clearly in checkout and seller dashboards.

19. Build Recommendation

Rands should launch as a trust-led marketplace with social features, not as a pure social app and not as a pure catalogue marketplace. The platform's strongest wedge is buyer confidence. Technically, that means the most important foundation is order-state governance, internal ledger design, and payout control.

If engineering capacity is limited, prioritise the following in order: seller onboarding and storefront, product catalogue, delivery quote workflow, Paystack payment collection and verification, internal ledger and payout states, admin policy toggles, and then feed, chat, and other social modules.

References

- [1] Paystack Developer Documentation - Split Payments.
- [2] Paystack Developer Documentation - Verify Payments.
- [3] Paystack Developer Documentation - Webhooks.
- [4] Paystack Developer Documentation - Accept Payments.
- [5] Paystack Developer Documentation - Payments overview.
- [6] Paystack Developer Documentation - Refunds.
- [7] Paystack Developer Documentation - Dedicated Virtual Accounts.
- <https://paystack.com/docs/payments/>